

büVpn8b

•oNö<¾<j<ôf Nf

rHg,ÿ V1.0

eåg ÿ 2026^t6g

r¶` ÿ kc_ SÑ^ rH

g,e#NöN:büVpn8b [Æetv,,•oNö<¾<j<ôf Nf

S T+g¶g,,<¾<j0 j!WW<æ~Æ<¾<j0 epcn~Óg,,SÊh8_Ã{—lŒ<ôf

vî_U

- 1 _Š
- 1.1 • Q™vîv„

1.2 • u(f Vô

1.3 g/<î[ŠNI
- 2 |û~ß`;OSg¶g„
- 2.1 g¶g„i,•đ

2.2 b€g/h

2.3 vî_U~Óg„
- 3 h8_Ãj!WW<¾<ç
- 3.1 N;cŠj!WW (Main)

3.2 N<Nö|û~ß (Event System)

3.3 epcn{it Vh (datamgr)

3.4 s©[¶epcn (PlayerData)

3.5 büVp{it Vh (picmgr)

3.6 bÖbý|û~ß (DraggableGridItem)

3.7 SarG|û~ß (card)

3.8 •¹hF|û~ß (BorderSpriteLoader)
- 4 U|j!WW<¾<ç
- 4.1 Wúx@z—OS (frmbase)

4.2 zà„• béuL—b (frm_chapter)

4.3 n8b uL—b (frm_game)

4.4 <¾•nuL—b (frm_setup)
- 5 epcn'M•n
- 5.1 zà„, 'M•n (DrChapter)

5.2 QsSa'M•n (DrLevel)

5.3 Tables 'M•n—ÆT
- 6 h8_Ã{—IÖ
- 6.1 büVpbsNq{—IÖ

6.2 •P• ~ÄhÀmK{—IÖ

6.3 OM•nNæcb{—IÖ

6.4 €ÜR)hÀmK{—IÖ

6.5 •¹hFR7e°{—IÖ
- 7 epcnc NES
- 7.1 [Xhc|û~ß

7.2 OSR>|û~ß
- 8 •Đ^Ls¯Xf
- 8.1 xINö%•IB

8.2 •oNö%•IB
- D_UA [Æetn•x - Main.cs

–D_UB [Æetn•x - picmgr.cs

–D_UC [Æetn•x - DraggableGridItem.cs

–D_UD [Æetn•x - PlayerData.cs

1.1.Š™vŭv,,

g,e#hceèW(Qh—bcĭ•đbŭVpŋ8b •oNöv,,gſſg,,<¾<ı0 j!WWR R 0 h8_Ă{—lŎSÊ[žs°~Æ,,ŷ OŶN:•oNö_ SŇ0 mK<ŎSÊ~ôbŧv,,S

1.2 • u(f Vô

g,•oNöf/N k>WúNŽ Unity _ dĭ_ SŇv,,bŭVpŋ8b ŷ •Đ^LNŽ iOS TÆ Android yŭR^ˆsSđ0 s©[ſſ• •ÇbŎbŷxŽrG[Æb bŭVpŷ n8b

1.3 g/ı[ŠNI

- bŭVpxŽrGŷ Piece / GridCellŷ ŷ g,,b [ÆetVprGv,,g \ SUQCŷ kĭN*xŽrG[ŭ^ˆVprGv,,N N*wé_bS:Wßŷ Qwg UV WPh TÆO
- •P• ~Ăŷ Connected Groupŷ ŷ W(bŭVp•Qh<N-ŷ OM•nvø•»N •;•‘Qs|ŭkcxnŷ,,xŽrG—ÆT ŷ SietOSbŎbŷyŭRˆ0
- OSR>ŷ Powerŷ ŷ s©[ſſ•Ŭ^Ln8b b@— m^€ v,,•Dn•ŷ →eö•ô•êRˆˆbY 0
- QsSaŷ Levelŷ ŷ bŭVpŋ8b v,,Wúg,SUOMŷ S T+_...büT v,,VprGSÊR Rr^LR SÂep0
- zà,,ŷ Chapterŷ ŷ QsSav,,—ÆT ŷ T N zà,,Q...v,,QsSaQqN«N _ ~,,%oÈVprG0
- Lubanŷ šŎeHv,,‘M•n^h]ăQwŷ Sĭ\ Excel ‘M•n[ŭQúN: JSON h<_ O>n8b <ŭSŎ0
- UV WPh ŷ ~1t WPh |ŭ~ßŷ f VôN: [0,1]ŷ u(NŽcšR6 RawImage f>y:SŶVpv,,TêN*S:Wß0

2.1 ũ~ß`i Qsgſg,,

g,n8b 'Çu(WúNŽ Unity _ dſv,,~ÄNöS gſg,,<¾<jſy etOSR N:N Y'\B~ſſy ^hs^\Bſy UI j!WWſy 0 •;•\Bſy h8_Än8b •;•ſy TÆepcn\B
^hs^\B ••#b@g u(b7uL—bN•N'ſy S bizà,,• béuL—b0 n8b uL—b0 <¾•nuL—b{I0 T uL—b~çb••ê~ßN v,, frmbase Wú|ſy ••
T j!WWNK••• ÇN<Nö|ũ~ßſy Event Systemſy [žs°g~€&T • Oá0 N<Nö|ũ~ß'Çu(%Â[ß€ j!_ ſy j!WWSê— Qs_Ä•ê]ñlèQÆv,,N<

2.2 b€g/h

n8b _ dſſy Unity 2022.3 (LTS)
• g,<ſſ ſy C# (.NET Standard 2.1)
R``u;_ dſſy DOTween (DG.Tweening)
JSON ^•R S ſy LitJson
'M•n^hhFgſſy Luban
UI |ũ~ßſy UGUIſy Unity Q...•n UI |ũ~ßſy
e‡g,n2gÓſy TextMeshPro
vîh ^sSäſy iOS / Android
rHg,cſR6ſy Git

2.3 vî_U~Óg,,

```
Assets/  
% % % Scenes/  
% % % SampleScene.unity # N;W:fo  
% % % Scripts/  
% % % Main.cs # n8b QeSäſy SU0<ſy N<Nö|ũ~ß  
% % % card.cs # zà,, SarLſy •û•lR``u;ſy  
% % % picmgr.cs # bÜVp{it Vhſy h8_Ä•;•'ſy  
% % % datamgr.cs # epcn{it Vhſy LubanR •}ſy  
% % % PlayerData.cs # s@[ſepcnſy c NES ſy  
% % % DraggableGridItem.cs # bÖbſy|ũ~ßſy •p• ~ÄbÖR``ſy  
% % % BorderSpriteLoader.cs # •¹hF•4VpR •}R7e°  
% % % DebugManager.cs # Æ <Öj!_ _ Qs  
% % % recttools.cs # wé_b]âQw|{  
% % % goldplay.cs / getpos.cs # •...R©• g,  
% % % ui/  
% % % % % frmbase.cs # UIz—OSWú|{  
% % % % % frm_chapter.cs # zà,,• béuL—b  
% % % % % frm_game.cs # n8b uL—b  
% % % % % frm_setup.cs # <¾•nuL—b  
% % % Config/ # Luban•êR``u b  
% % % % % DrChapter.cs / DrLevel.cs  
% % % % % Tables.cs / Tb*.cs  
% % % % % Extensions/ # bi\U]âQw  
% % % Plugins/  
% % % % % DOTween/ # R``u;cÖNö  
% % % % % LitJson/ # JSON^"  
% % % Resources/  
% % % Card/ (lt,t,rt,r,rb,b,lb,l) # •¹hF•4Vp  
% % % GridCell / levelpic # ~,,R60S  
% % % *.jpg / *.png # QsSaVprG
```

3.1 N;csj!WW (Main.cs)

Main {f/n8b T/R`QeSãÿ ••#R YËS Qh\@~ÄNö0 lèQÇENçNö0 {it n8b r¶` 0 ‘Çu(SUOçÿ Singletonÿ j!_ÿ ••Ç Main.inst Qh N;%•€L•#ÿ

1. R YËS DOTween R`u;_ dÎ
2. cÐO>Qh\@NçNö|û~ßÿ RegistEvent / DispEvent / UnRegistEventÿ
3. {it n8b f,PV`bY r¶` ÿ ••Ç Time.timeScale c§R6ÿ
4. cÐO>QsSa'íe°_ YËRÿ€ÿÿ m^€ 10 OSR>ÿ
5. R YËS n8b epcnÿ n –dc NES vî_Uÿ

```
/// <summary>
/// Main.cs - n8b N;csj!WW
/// €L•#ÿ n8b R YËS 0 NçNö|û~ß0 f,P\{it
/// </summary>
public class Main : MonoBehaviour
{
    public delegate object registfun(object parm);

    public static Main inst;
    public Material matboader;
    public GameObject clickEffectPrefab;

    // f,P\r¶`
    private static bool isGamePaused = false;
    public static bool IsPaused
    {
        get => isGamePaused;
        set => isGamePaused = value;
    }

    // NçNö|û~ß[¹Vh
    static Dictionary<string, List<registfun>> evs = new();

    public static object DispEvent(string ev, object parm = null)
    {
        if (evs.ContainsKey(ev))
        {
            for (int i = 0; i < evs[ev].Count; i++)
            {
                var p = evs[ev][i](parm);
                if (p != null) return p;
            }
            return null;
        }
        else
        {
            Debug.LogError("lig Y t m^`o" + ev);
            return null;
        }
    }

    public static void RegistEvent(string ev, registfun fun)
    {
        if (!evs.ContainsKey(ev))
            evs[ev] = new List<registfun>();
        evs[ev].Add(fun);
    }

    public static void UnRegistEvent(string ev, registfun fun)
    {
        if (evs.ContainsKey(ev))
            evs[ev].Remove(fun);
    }

    internal static void SendEvent(string v) => DispEvent(v);

    private void Awake()
```


3.2 N<Nö|û-ß (Event System)

û-ßN [šNiv„Qh\@N<Nöy

- gamebegin - n8b _ YĖÿ %æSÑzà,,uL—b™—k!f>y:
- level_play - •ÛQeQsSaÿ d: ^& levelld SÂepÿ •ÔVp 1/0
- level_next - R cbR0N N Qs
- level_back - •ÔVp zà,,• béuL—b
- show_setup / show_next - f>y:<¾•n/N N QsuL—b
- event_msg - m``ocĐy:ÿ Y,OSR>N •³ÿ
- event_tips - —¾^!cĐy:ÿ Y,Vđ—¾j!_ÿ
- onLevelChange - QsSa•Û^!SØS eö•êR``OÝ[X[Xhc
- onpowerChange - OSR>P<SØS eö•êR``OÝ[X[Xhc
- onChapterChange - zà,,R cbeöR7e°uL—b

```
// N<Nö|û-ß m>SÑy:0<
// frm_chapter.cs N-v„N<Nö|û-ß
Main.RegisterEvent("gamebegin", (x) =>
{
    brushChapterContent();
    show();
    UpdateStaminaDisplay();
    return 1;
});

// frm_game.cs N-v„N<Nö|û-ß
Main.RegisterEvent("level_play", (x) =>
{
    if (!PlayerData.gd.hasEnoughpower(10))
    {
        Main.DispEvent("event_msg", "powerN •³ÿ eàlô_ YĖn8b ");
        return 0;
    }
    PlayerData.gd.m^€ power(10);
    var leevel = datamgr.Instance.GetLevel((int)x);
    show();
    StartCoroutine(load(leevel));
    return 1;
});

// æSÑN<Nöy:0<
Main.DispEvent("level_play", PlayerData.gd.levelld);
Main.SendEvent("level_next");
Main.DispEvent("show_setup");
```

3.3 epcn{it Vh (datamgr.cs)

datamgr.cs - epcn{it Vh (datamgr.cs) Çu(SUO<j!_ÿ••Ç datamgr.Instance Qh\@<¿•î0 WúNŽ Luban 'M•n^hhFg
e/c N\$yIR -je1_ÿ1ÿ••j!_N_ÿ••ÇW(Inspector N bÖby TextAsset [ùEavôc¥R-}ÿ2ÿ•Đ~Lj!_N_ÿNî Assets/data/ vî_Uk

```

/// <summary>
/// datamgr.cs - epcn{it Vh
/// €L•#ÿ R •}Luban'M•n^hÿ cĐ0>epcn<¿•ic¥Sä
/// </summary>
public class datamgr : MonoBehaviour
{
    [Header("epcne†Nö")]
    public TextAsset tbchapter;
    public TextAsset tblevel;

    private static datamgr _instance;
    public static datamgr Instance
    {
        get
        {
            if (_instance == null)
            {
                var x = Instantiate(Resources.Load("DataManager")) as GameObject;
                _instance = x.GetComponent<datamgr>();
            }
            return _instance;
        }
    }

    public Tables Tables { get; private set; }
    public bool IsLoaded { get; private set; } = false;

    private void Awake()
    {
        if (_instance == null)
        {
            _instance = this;
            DontDestroyOnLoad(gameObject);
            LoadConfigTables();
        }
        else if (_instance != this)
            Destroy(gameObject);
    }

    public void LoadConfigTables()
    {
        Debug.Log("_ YÉR •}'M•n^h...");
        try
        {
            System.Func<string, JSONNode> loader = (string file) =>
            {
                TextAsset dataAsset = null;
                switch (file)
                {
                    case "tbchapter": dataAsset = tbchapter; break;
                    case "tblevel": dataAsset = tblevel; break;
                }
                if (dataAsset != null && !string.IsNullOrEmpty(dataAsset.text))
                    return JSON.Parse(dataAsset.text);

                string filePath = Path.Combine(
                    Application.dataPath, "data", file + ".json");
                if (File.Exists(filePath))
                    return JSON.Parse(File.ReadAllText(filePath));

                Debug.LogError("'M•ne†NöN [XW(: " + filePath);
                return null;
            };

            Tables = new Tables(loader);
            IsLoaded = true;
        }
    }
}

```


3.4 s@ [Tepon (PlayerData.cs)

PlayerData - 00C1M8b (PlayerData.cs) bî_SRMQsSa ID0]ò%ã• QsSa-ÆT 0 _SRMzà,, ID0 OSR>P<SÊOSF
epcn'Çu(JSON h<_ c NES R0 Application.persistentDataPath/playerData.json0 n8b T/R" eö•êR" R•jy epcnSØföeö••Ç

```

/// <summary>
/// GameData - n8b •Ð^Leöepcnj!W<
/// </summary>
public class GameData
{
    JsonData data;
    opend op; // ]ò%ã• -ÆT

    public GameData(JsonData _data)
    {
        data = _data;
        op = new opend(openeddata);
    }

    // _SRMQsSaID
    internal int levelid
    {
        get => int.Parse(data.Has("levelid")
            ? data["levelid"].ToString() : "100001");
        set { data["levelid"] = value.ToString();
            Main.DispEvent("onLevelChange"); }
    }

    // _SRMzà,,ID
    internal int currChapter
    {
        get => int.Parse(data.Has("currChapter")
            ? data["currChapter"].ToString() : "1");
        set { data["currChapter"] = value.ToString();
            Main.DispEvent("onChapterChange"); }
    }

    // OSR>P<ÿ T+•êR" `bY <i{-ÿ
    internal int power
    {
        get
        {
            int cur = int.Parse(data.Has("power")
                ? data["power"].ToString() : "100");
            long last = lastpowerUpdateTime;
            long now = DateTime.Now.Ticks / TimeSpan.TicksPerSecond;
            long elapsed = now - last;
            int recovered = (int)(elapsed / (15 * 60)); // 15R "ÿ/p¹
            if (recovered > 0)
            {
                cur = Math.Min(100, cur + recovered);
                data["power"] = cur.ToString();
                lastpowerUpdateTime = now;
            }
            return cur;
        }
        set
        {
            data["power"] = Math.Min(100, value).ToString();
            lastpowerUpdateTime = DateTime.Now.Ticks
                / TimeSpan.TicksPerSecond;
            Main.DispEvent("onpowerChange");
        }
    }

    internal long lastpowerUpdateTime
    {
        get => long.Parse(data.Has("lastpowerUpdateTime")
            ? data["lastpowerUpdateTime"].ToString()
            : (DateTime.Now.Ticks / TimeSpan.TicksPerSecond).ToString());
        set => data["lastpowerUpdateTime"] = value.ToString();
    }
}

```


3.5 büVp{it Vh (picmgr.cs)

3.5.1 VpŋGR RŋN xŽrGR ^ú

h9cn DrLevel 'M•nN-v,, LevelFigureXŷ R epŷ TŒ LevelFigureYŷ ^Lepŷ ŷ \ SŶVpWGS R RŋN: width×height N*xŽrG0 kŊN anchoredPosition [šOM0

```

/// <summary>
/// picmgr.cs - büVp{it Vhŷ h8_Ä•;•'ŷ
/// </summary>
public class picmgr : MonoBehaviour
{
    public Texture2D pic;
    public int width = 3;
    public int height = 3;

    internal static picmgr instance;
    DrLevel curlevel;

    // kŊN*xŽrGv,,[½/š0
    internal float carWid => trans.rect.width / width;
    internal float carHei => trans.rect.height / height;

    RectTransform trans => GetComponent<RectTransform>();

    private void Awake()
    {
        instance = this;
        ResizeChapterContent();
    }

    public void ResizeChapterContent()
    {
        // c 9:16k00<• ^"\0^U
        float margin = 100f;
        float availableWidth = Screen.width - 2 * margin;
        float availableHeight = Screen.height - 2 * margin;
        float targetWidth, targetHeight;

        if (availableWidth / availableHeight > 9f / 16f)
        {
            targetHeight = availableHeight;
            targetWidth = targetHeight * 9f / 16f;
        }
        else
        {
            targetWidth = availableWidth;
            targetHeight = targetWidth * 16f / 9f;
        }

        float wside = (Screen.width - targetWidth) / 2;
        float hside = (Screen.height - targetHeight) / 2;
        trans.offsetMin = new Vector2(wside, hside);
        trans.offsetMax = new Vector2(-wside, -hside);
    }

    /// <summary>
    /// R •}QsSaŷ <%•n^LR ep0 R •}VpŋG0 R ^úxŽrG
    /// </summary>
    public IEnumerator LoadLevel(DrLevel leevel)
    {
        curlevel = leevel;
        width = leevel.LevelFigureX;
        height = leevel.LevelFigureY;
        pic = Resources.Load(leevel.LevelFigure) as Texture2D;

        if (leevel.DifficultyTier == 2)
            Main.DispEvent("event_tips", "Vð-¾j!_ ");
    }
}

```


•1hF|û~ßN:kîN*xŽrGR ^ú 8 N*[P[ùEaÿ VÛN*•1ÿ Top/Bottom/Left/RightBorderÿ TÆVÛN*%Öÿ TopLeft/TopRight/Bottom
BorderSpriteLoader ~ÄNöNî Resources/Card vî_UR •}0

```
/// <summary>
/// R ^úxŽrGv„N][«h<•1hF
/// </summary>
private void CreateBorders(GameObject cellObject,
                           float cellWidth, float cellHeight)
{
    // n -deç•1hF
    for (int i = cellObject.transform.childCount - 1; i >= 0; i--)
        Destroy(cellObject.transform.GetChild(i).gameObject);

    float borderWidth = 20f;
    float bw = Mathf.Min(borderWidth,
        cellWidth / 2f, cellHeight / 2f);

    // VÛN*%Ö
    CreateBorder(cellObject, "TopLeftBorder",
        new Vector2(0, cellHeight - bw), new Vector2(bw, bw));
    CreateBorder(cellObject, "TopRightBorder",
        new Vector2(cellWidth - bw, cellHeight - bw), new Vector2(bw, bw));
    CreateBorder(cellObject, "BottomRightBorder",
        new Vector2(cellWidth - bw, 0), new Vector2(bw, bw));
    CreateBorder(cellObject, "BottomLeftBorder",
        new Vector2(0, 0), new Vector2(bw, bw));

    // VÛN*•1
    CreateBorder(cellObject, "TopBorder",
        new Vector2(bw, cellHeight - bw),
        new Vector2(Mathf.Max(0f, cellWidth - 2f * bw), bw));
    CreateBorder(cellObject, "BottomBorder",
        new Vector2(bw, 0),
        new Vector2(Mathf.Max(0f, cellWidth - 2f * bw), bw));
    CreateBorder(cellObject, "LeftBorder",
        new Vector2(0, bw),
        new Vector2(bw, Mathf.Max(0f, cellHeight - 2f * bw)));
    CreateBorder(cellObject, "RightBorder",
        new Vector2(cellWidth - bw, bw),
        new Vector2(bw, Mathf.Max(0f, cellHeight - 2f * bw)));
}

private void CreateBorder(GameObject parent, string name,
    Vector2 anchoredPosition, Vector2 sizeDelta)
{
    GameObject border = new GameObject(name);
    border.transform.SetParent(parent.transform, false);

    RectTransform rect = border.AddComponent<RectTransform>();
    rect.anchorMin = new Vector2(0, 0);
    rect.anchorMax = new Vector2(0, 0);
    rect.pivot = new Vector2(0, 0);
    rect.anchoredPosition = anchoredPosition;
    rect.sizeDelta = sizeDelta;

    Image img = border.AddComponent<Image>();
    img.color = Color.white;

    BorderSpriteLoader loader =
        border.AddComponent<BorderSpriteLoader>();
    loader.resourceName = MapBorderNameToResource(name);
    loader.LoadSprite();
}

private string MapBorderNameToResource(string name)
{
    return name switch
    {
        "TopLeftBorder"    => "lt",
        "TopBorder"        => "t",
    }
}
```


•¹hFR7e°{—lÖR`` fôe°xŽrGv„N][«h<•¹hFf>y:0 {—lÖR N:N\$-ÄrizËY t ÿ —^bÖbý~Äÿ SÂepeàSÂ‘í•}ÿ TÆbÖbý~Äÿ SÂep
h8_Ã•;•ÿ WÚNŽxŽrG_SRMv„[Ž-EOM•nÿ anchoredPositionÿ ÿ <i{—VÛN*e¹T •»\Ev„~„g OM•nÿ W(~ÄQ...d }"«åOM•nf/T&

```
/// <summary>
/// fôe°•¹hFÿ —^bÖbý~Äÿ
/// </summary>
public void UpdateBorderVisibility()
{
    List<DraggableGridItem> allItems = new List<DraggableGridItem>();
    foreach (Transform child in transform)
    {
        var item = child.GetComponent<DraggableGridItem>();
        if (item != null && !item.IsBeingDragged)
            allItems.Add(item);
    }
    if (allItems.Count > 0)
        UpdateBorderVisibility(allItems);
}

/// <summary>
/// fôe°•¹hFÿ c [š-Äÿ u(NŽbÖbý~Äb —^bÖbý~Äÿ
/// </summary>
public void UpdateBorderVisibility(List<DraggableGridItem> group)
{
    if (group == null || group.Count == 0) return;

    RectTransform rectTransform = GetComponent<RectTransform>();
    Vector2 rectSize = rectTransform.rect.size;
    float cellWidth = rectSize.x / width;
    float cellHeight = rectSize.y / height;
    float checkThreshold = 1.5f;

    foreach (DraggableGridItem item in group)
    {
        RectTransform itemRect = item.GetComponent<RectTransform>();
        if (itemRect == null) continue;

        // %ãg••;•¹WPh
        string[] parts = item.gameObject.name
            .Replace("GridCell_", "").Split('_');
        if (parts.Length != 2) continue;
        int x = int.Parse(parts[0]);
        int y = int.Parse(parts[1]);

        // VÛe¹T •»\EhÀmK
        Vector2 eRight = itemRect.anchoredPosition
            + new Vector2(cellWidth, 0);
        Vector2 eLeft = itemRect.anchoredPosition
            + new Vector2(-cellWidth, 0);
        Vector2 eTop = itemRect.anchoredPosition
            + new Vector2(0, cellHeight);
        Vector2 eBottom = itemRect.anchoredPosition
            + new Vector2(0, -cellHeight);

        var rN = FindGridItemAtAnchoredPosition(eRight, group, checkThreshold);
        var lN = FindGridItemAtAnchoredPosition(eLeft, group, checkThreshold);
        var tN = FindGridItemAtAnchoredPosition(eTop, group, checkThreshold);
        var bN = FindGridItemAtAnchoredPosition(eBottom, group, checkThreshold);

        // [Û%Ô~¿•»\E
        Vector2 eTR = itemRect.anchoredPosition
            + new Vector2(cellWidth, cellHeight);
        Vector2 eTL = itemRect.anchoredPosition
            + new Vector2(-cellWidth, cellHeight);
        Vector2 eBR = itemRect.anchoredPosition
            + new Vector2(cellWidth, -cellHeight);
        Vector2 eBL = itemRect.anchoredPosition
            + new Vector2(-cellWidth, -cellHeight);
    }
}
```


bSNq{—lŒO•u(xn[š'—•g:yÍ[Pÿ seed = levelId × 1000 + (isHard ? 1 : 0)ÿ ÿ xnOÝT N QsSakİk!bSNq~ŒgœN •ô0 {—lŒmA

```
/// <summary>
/// bSNqxŽrGOM•n
/// </summary>
public void ShuffleGridPositions(int level = 1,
    int maxKeepCount = -1, bool isHard = false)
{
    List<RectTransform> children = new List<RectTransform>();
    List<Vector2> originalPositions = new List<Vector2>();
    List<Vector2Int> originalGridCoords = new List<Vector2Int>();

    foreach (Transform child in transform)
    {
        var rect = child.GetComponent<RectTransform>();
        if (rect == null) continue;
        children.Add(rect);
        originalPositions.Add(rect.anchoredPosition);

        string[] parts = child.name.Replace("GridCell_", "").Split('_');
        if (parts.Length == 2 && int.TryParse(parts[0], out int x)
            && int.TryParse(parts[1], out int y))
            originalGridCoords.Add(new Vector2Int(x, y));
        else
            originalGridCoords.Add(new Vector2Int(-1, -1));
    }

    if (children.Count < 2) return;
    Vector2 cellSize = children[0].sizeDelta;

    List<int> availableIndices = new List<int>();
    for (int i = 0; i < children.Count; i++)
        availableIndices.Add(i);

    // xn[šyÍ[P
    int seed = level * 1000 + (isHard ? 1 : 0);
    System.Random rng = new System.Random(seed);

    // OÝuYSarLep'İ
    if (maxKeepCount == -1)
        maxKeepCount = Mathf.Max(0,
            Mathf.RoundToInt(children.Count * 0.3f));
    maxKeepCount = Mathf.Clamp(maxKeepCount,
        0, children.Count - 1);
    int cardsToKeep = rng.Next(0, maxKeepCount + 1);

    // -•g:• bŒOÝuY~y
    List<int> allIndices = new List<int>();
    for (int i = 0; i < children.Count; i++)
        allIndices.Add(i);

    System.Random shuffleRng = new System.Random(seed + 1);
    for (int i = allIndices.Count - 1; i > 0; i--)
    {
        int j = shuffleRng.Next(0, i + 1);
        (allIndices[i], allIndices[j]) =
            (allIndices[j], allIndices[i]);
    }

    List<int> keepIndices = allIndices.GetRange(0, cardsToKeep);
    foreach (int k in keepIndices)
        availableIndices.Remove(k);

    List<int> targetIndices = new List<int>(availableIndices);

    // VŒ-‰j!_ Ÿ •Ýy»0 S
    if (isHard && originalGridCoords.Count == children.Count)
    {
        System.Random distRng = new System.Random(seed + 3);
        for (int attempt = 0; attempt < 3; attempt++)
```


•MS†b@g xŽrGŷ [ùkîN*xŽrGhÀgâQvVÛN*e¹T v„[ž–E•»\E0 Y,gœekîN*e¹T •ý[XW(N T yđS9'Mkcxny R R\$[šbüVp[Æb 0 €

```
internal void CheckSucess()
{
    bool suc = true;
    foreach (Transform child in transform)
    {
        var c = child.GetComponent<DraggableGridItem>();
        if (c == null) continue;

        string[] parts = child.name.Replace("GridCell_", "").Split('_');
        if (parts.Length != 2) continue;
        int x = int.Parse(parts[0]);
        int y = int.Parse(parts[1]);

        DraggableGridItem rN = null, lN = null, tN = null, bN = null;
        RectTransform cr = child.GetComponent<RectTransform>();

        foreach (Transform other in transform)
        {
            if (other == child) continue;
            var oi = other.GetComponent<DraggableGridItem>();
            if (oi == null) continue;

            Vector2 d = other.GetComponent<RectTransform>()
                .anchoredPosition - cr.anchoredPosition;
            float t = 1f;
            if (Mathf.Abs(d.x - cr.sizeDelta.x) < t
                && Mathf.Abs(d.y) < t) rN = oi;
            else if (Mathf.Abs(d.x + cr.sizeDelta.x) < t
                && Mathf.Abs(d.y) < t) lN = oi;
            else if (Mathf.Abs(d.y - cr.sizeDelta.y) < t
                && Mathf.Abs(d.x) < t) tN = oi;
            else if (Mathf.Abs(d.y + cr.sizeDelta.y) < t
                && Mathf.Abs(d.x) < t) bN = oi;
        }

        if (!(rN == null || rN.name == $"GridCell_{x+1}_{y}")
            && (lN == null || lN.name == $"GridCell_{x-1}_{y}")
            && (tN == null || tN.name == $"GridCell_{x}_{y+1}")
            && (bN == null || bN.name == $"GridCell_{x}_{y-1}"))
        { suc = false; break; }
    }

    if (suc)
    {
        var t = transform.GetComponent<RectTransform>();
        Vector2 cur = t.anchoredPosition;
        t.DOAnchorPos(new Vector2(cur.x, cur.y + 100f), 0.5f)
            .OnComplete(() =>
            {
                for (int i = 0; i < transform.childCount; i++)
                    transform.GetChild(i)
                        .GetComponent<DraggableGridItem>().enabled = false;
                Main.DispEvent("show_next", cur.level);
            });
    }
}
```


3.6 bÖbýlû~ß (DraggableGridItem.cs)

3.6.1 c¥Säřžs°N \^~[šNl

```
public class DraggableGridItem : MonoBehaviour,
    IBeginDragHandler, IDragHandler, IEndDragHandler
{
    public RectTransform rectTransform => GetComponent<RectTransform>();
    public Canvas canvas;

    private static int originalPositionIndex; // •wYĖOM•n}"_
    public static int targetPositionIndex;    // vîh OM•n}"_
    public int PositionIndex { get; set; }     // _SRMOM•n}"_

    private bool isDragging = false;
    public bool IsBeingDragged => isDragging;
    public static bool isAnyItemDragging = false;

    // •»c¥h _xÿ u1•¹hFR7e°{-lŎ<¾•nÿ
    public bool adjacentLeft = false;
    public bool adjacentRight = false;
    public bool adjacentTop = false;
    public bool adjacentBottom = false;

    // u(NŽ•û•LR``u;TĖUV
    internal Texture2D pic;
    internal float uvX, uvY, uvWidth, uvHeight;

    // OM•n\^`
    static Vector2 originalPosition => new Vector2(
        (originalPositionIndex / wid) * carWid,
        (originalPositionIndex % hei) * carHei);
    static Vector2 targetPosition => new Vector2(
        (targetPositionIndex / wid) * carWid,
        (targetPositionIndex % hei) * carHei);
    Vector2 vecPosition => new Vector2(
        (PositionIndex / wid) * carWid,
        (PositionIndex % hei) * carHei);

    // •Qh<\:[øÿ YŎbXpicmgrÿ
    public static float carWid => picmgr.instance.carWid;
    public static float carHei => picmgr.instance.carHei;
    public static int wid => picmgr.instance.width;
    public static int hei => picmgr.instance.height;

    // bÖbý-Ä
    private List<DraggableGridItem> dragGroup = null;
    private Dictionary<DraggableGridItem, Vector2>
        groupOriginalPositions = null;
    private Dictionary<DraggableGridItem, int>
        groupOriginalSiblingIndices = null;

    Transform parent => transform.parent;
    RawImage rawImage => GetComponent<RawImage>();
}
```

3.6.2 OnBeginDrag - _ YĖbÖbý

_ YĖbÖbýeö<°_UxŽřGOM•n}"_ TĖ\B~šÿ Ė u(CollectConnectedGroup e6-Æ•Ĭ• ~Äÿ \ ~ÄQ...b@g b TXcĐSG•óg N VE

```
public void OnBeginDrag(PointerEventData eventData)
{
    if (isAnyItemDragging) return;
    isAnyItemDragging = true;

    originalPositionIndex = PositionIndex;
    originalSiblingIndex = transform.GetSiblingIndex();
}
```


<i{—OMyû'İ^vPZ•¹uL—PR6ÿ € †QetN*bÖbÿ~Äv„b@g b TXÿ 0 ^"u(OM•nT R7e°N\$~Ä•¹hFÿ —^bÖbÿ~ÄTÆbÖbÿ~ÄR R+

```
public void OnDrag(PointerEventData eventData)
{
    RectTransform parentRect = transform.parent as RectTransform;
    if (parentRect == null) return;

    // <i{-SÿYËOMyû
    Vector2 rawNewPos = rectTransform.anchoredPosition
        + eventData.delta / canvas.scaleFactor;
    Vector2 rawDelta = rawNewPos - originalPosition;

    // •¹uL—PR6ÿ € †Q~ÄQ...b@g b TXÿ
    List<DraggableGridItem> allItems = dragGroup
        ?? new List<DraggableGridItem>() { this };

    float minDX = float.MinValue, maxDX = float.MaxValue;
    float minDY = float.MinValue, maxDY = float.MaxValue;

    foreach (var item in allItems)
    {
        Vector2 orig;
        if (item == this) orig = originalPosition;
        else if (groupOriginalPositions != null
            && groupOriginalPositions.TryGetValue(item, out Vector2 o))
            orig = o;
        else continue;

        float iw = item.rectTransform.rect.width;
        float ih = item.rectTransform.rect.height;

        minDX = Mathf.Max(minDX, -orig.x);
        maxDX = Mathf.Min(maxDX,
            parentRect.rect.width - iw - orig.x);
        minDY = Mathf.Max(minDY, -orig.y);
        maxDY = Mathf.Min(maxDY,
            parentRect.rect.height - ih - orig.y);
    }

    Vector2 clampedDelta = new Vector2(
        Mathf.Clamp(rawDelta.x, minDX, maxDX),
        Mathf.Clamp(rawDelta.y, minDY, maxDY));

    // ^"u(OM•n
    rectTransform.anchoredPosition =
        originalPosition + clampedDelta;

    if (dragGroup != null && groupOriginalPositions != null)
    {
        foreach (var it in dragGroup)
        {
            if (it == this) continue;
            if (groupOriginalPositions.TryGetValue(it,
                out Vector2 orig))
                it.rectTransform.anchoredPosition =
                    orig + clampedDelta;
        }
    }

    // R7e°N$~Ä•¹hF
    if (picmgr.instance != null)
    {
        picmgr.instance.UpdateBorderVisibility();
        if (dragGroup != null)
            picmgr.instance.UpdateBorderVisibility(dragGroup);
    }

    // <i{-vîh OM•n}"_
    var diff = rectTransform.anchoredPosition - vecPosition;
    if (Mathf.Abs(diff.x) > carWid / 2
```


R\$e-f/T&ná³NæcbgaNöy OM•nN T N OM•nT lŒy y b RŸR bg^LOM•nNæcbÿ Y1•%R etOS• VpSŸOM0

```
public void OnEndDrag(PointerEventData eventData)
{
    bool shouldSwap = false;

    if (originalPositionIndex != targetPositionIndex)
    {
        bool isPositionValid = CheckPotentialPositionValidity();
        if (isPositionValid) shouldSwap = true;
    }

    if (shouldSwap) SwapPositions(targetPosition);
    else ResetAllDraggedItemsToOriginalPosition();

    dragGroup = null;
    groupOriginalPositions = null;
    groupOriginalSiblingIndices = null;
    isDragging = false;
}

private bool CheckPotentialPositionValidity(
    Vector2 potentialTargetPos = default)
{
    List<DraggableGridItem> list = (dragGroup != null
        && dragGroup.Count > 0)
        ? new List<DraggableGridItem>(dragGroup)
        : new List<DraggableGridItem>() { this };

    RectTransform parentRect = transform.parent as RectTransform;
    if (parentRect == null) return false;

    Rect parentBounds = new Rect(0, 0,
        parentRect.rect.width, parentRect.rect.height);

    foreach (var item in list)
    {
        Vector2 newPos = item.rectTransform.anchoredPosition
            + new Vector2(item.rectTransform.sizeDelta.x / 2,
                item.rectTransform.sizeDelta.y / 2);
        if (!parentBounds.Contains(newPos)) return false;
    }
    return true;
}
```

3.6.5 •P• ~ÄhÀmK (DFS)

O•u(mñ^iO QHd }"ÿ DFSÿ {—lŒy WúNžxžrGv,, adjacent h _x•Ū^L•MS†0 NŒ_SRMxžrG_ YËÿ l¿VŪN*e†T •_RbìlUÿ \ a

```
private void CollectConnectedGroup()
{
    dragGroup = new List<DraggableGridItem>();
    groupOriginalPositions = new Dictionary<...>();
    groupOriginalSiblingIndices = new Dictionary<...>();

    // g,,^úWPh f \
    Dictionary<(int, int), DraggableGridItem> map = new();
    foreach (Transform child in parent)
    {
        var item = child.GetComponent<DraggableGridItem>();
        if (item == null) continue;
        if (TryParseCellName(child.name, out int cx, out int cy))
            map[(cx, cy)] = item;
    }

    if (!TryParseCellName(this.name, out int startX, out int startY))
    {
        dragGroup.Add(this);
        groupOriginalPositions[this] = rectTransform.anchoredPosition;
    }
}
```


OM•nNꞑcb{—lŒŸ 1) <i{—bÖbŷ~Äv„vîh OM•nŷ 2) <ÆR+zzQúOM•nTÆ^«%†vÖxŽrGŷ 3) ^úzË%†vÖxŽrGR0zzQúOM•n

```
private void SwapPositions(Vector2 targetPos)
{
    PositionIndex = GetIndexFromAnchoredPosition(targetPos);

    var list = (dragGroup != null && dragGroup.Count > 0)
        ? new List<DraggableGridItem>(dragGroup)
        : new List<DraggableGridItem>() { this };

    // <°_USŸYËOM•nTÆ\B~$
    Dictionary<DraggableGridItem, Vector2> originals = new();
    Dictionary<DraggableGridItem, int> siblings = new();
    foreach (var item in list)
    {
        originals[item] = groupOriginalPositions != null
            && groupOriginalPositions.TryGetValue(item, out var p)
            ? p : item.rectTransform.anchoredPosition;
        siblings[item] = item.transform.GetSiblingIndex();
    }

    Vector2 srcPos = originals[this];
    Dictionary<DraggableGridItem, Vector2> targets = new();
    foreach (var item in list)
        targets[item] = targetPos + (originals[item] - srcPos);

    // <i{—zzQúOM•n
    List<Vector2> emptyPos = new();
    foreach (var kvp in originals)
    {
        bool filled = false;
        foreach (var t in targets.Values)
            if (Vector2.Distance(t, kvp.Value) < 1f)
                { filled = true; break; }
        if (!filled) emptyPos.Add(kvp.Value);
    }

    // <i{—^«%†vÖSarl
    List<DraggableGridItem> covered = new();
    foreach (Transform child in parent)
    {
        var item = child.GetComponent<DraggableGridItem>();
        if (item == null || list.Contains(item)) continue;
        foreach (var t in targets.Values)
        {
            if (Vector2.Distance(
                item.rectTransform.anchoredPosition, t) < 1f)
                { covered.Add(item); break; }
        }
    }

    // c'^^•T f \
    emptyPos.Sort((a, b) => {
        int yc = b.y.CompareTo(a.y);
        return yc != 0 ? yc : a.x.CompareTo(b.x);
    });
    covered.Sort((a, b) => {
        int yc = b.rectTransform.anchoredPosition.y
            .CompareTo(a.rectTransform.anchoredPosition.y);
        return yc != 0 ? yc :
            a.rectTransform.anchoredPosition.x
            .CompareTo(b.rectTransform.anchoredPosition.x);
    });

    Dictionary<DraggableGridItem, Vector2> replace = new();
    for (int i = 0; i < Mathf.Min(covered.Count, emptyPos.Count); i++)
        replace[covered[i]] = emptyPos[i];

    // bg^LR``u;
    Sequence seq = DOTween.Sequence();
```


3.7 SarG|û-ß (cardes)

urru upDââ, put-ßb/ÇqûßSarß) kĭ_ SarG[û^"N N*QsSaÿ h9cnQsSarĭ` f>y:N T Q...[4ÿ]ò• QsSarGf>y:zà,,,%ÈVp

```

public class card : MonoBehaviour
{
    public Texture2D back;
    internal int levelid;
    internal float uvX, uvY, uvWidth, uvHeight;
    internal Texture texture;
    public TextMeshProUGUI level;
    public bool isTurning = false;

    RawImage rawImage => GetComponent<RawImage>();

    internal void Load()
    {
        if (levelid < PlayerData.gd.levelid)
        {
            if (PlayerData.gd.isOpened(levelid))
            {
                rawImage.uvRect = new Rect(
                    uvX, (1 - uvY - uvHeight), uvWidth, uvHeight);
                rawImage.texture = texture;
                level.gameObject.SetActive(false);
            }
            else
            {
                DOVirtual.DelayedCall(1.0f, () =>
                {
                    rawImage.uvRect = new Rect(0, 0, 1, 1);
                    rawImage.texture = back;
                    level.gameObject.SetActive(false);
                    isTurning = true;
                    transform.DOScaleX(0, .5f).OnComplete(() =>
                    {
                        transform.localScale = new Vector3(0, 1, 1);
                        rawImage.uvRect = new Rect(
                            uvX, (1 - uvY - uvHeight), uvWidth, uvHeight);
                        rawImage.texture = texture;
                        transform.DOScaleX(1, 0.5f).OnComplete(() =>
                        {
                            transform.localScale = new Vector3(1, 1, 1);
                            isTurning = false;
                        });
                    });
                });
            }
        }
        else
        {
            rawImage.uvRect = new Rect(0, 0, 1, 1);
            rawImage.texture = back;
            level.gameObject.SetActive(true);
        }
    }
}

```

3.8.1hF|û-ß (BorderSpriteLoader.cs)

```

public class BorderSpriteLoader : MonoBehaviour
{
    public string resourceName;
    public bool keepColorIfMissing = true;
    private static Dictionary<string, Sprite> spriteCache;

    internal void LoadSprite()
    {
        if (string.IsNullOrEmpty(resourceName)) return;
        Image img = GetComponent<Image>();
        if (img == null) return;

        if (spriteCache == null)
            spriteCache = new Dictionary<string, Sprite>();

        if (spriteCache.TryGetValue(resourceName, out var s))
        {
            img.sprite = s; img.color = Color.white;
        }
        else
        {
            s = Resources.Load<Sprite>("Card/" + resourceName);
            if (s != null)
            {
                img.sprite = s; img.color = Color.white;
                spriteCache[resourceName] = s;
            }
            else
            {
                img.sprite = null;
                img.color = keepColorIfMissing ? Color.gray : Color.clear;
            }
        }
    }

    public void SetResourceAndLoad(string key)
    {
        resourceName = key;
        LoadSprite();
    }

    internal void RefreshBasedOnFlags(string prop,
        bool isTopCorrect, bool isLeftCorrect,
        bool isRightCorrect, bool isBottomCorrect,
        bool isConnerCorrect)
    {
        string key = resourceName;
        switch (prop)
        {
            case "lt":
                if (isLeftCorrect && isTopCorrect && !isConnerCorrect)
                    key = "lt_revert";
                else if (!isLeftCorrect && !isTopCorrect) key = "lt";
                else if (isLeftCorrect && !isTopCorrect) key = "t";
                else if (!isLeftCorrect && isTopCorrect) key = "l";
                break;
            case "rt":
                if (isRightCorrect && isTopCorrect && !isConnerCorrect)
                    key = "rt_revert";
                else if (!isRightCorrect && !isTopCorrect) key = "rt";
                else if (isRightCorrect && !isTopCorrect) key = "t";
                else if (!isRightCorrect && isTopCorrect) key = "r";
                break;
            case "rb":
                if (isRightCorrect && isBottomCorrect && !isConnerCorrect)
                    key = "rb_revert";
                else if (!isRightCorrect && !isBottomCorrect) key = "rb";
        }
    }
}

```


4.1 UI (frmbase.cs)

frmbase f/b@g UI z—OSv„Wú|{ÿ cÐO>~ßN v„ show/hide u T}Thg 0 • •Ç transform.Find("Root") gåb~h9„p¹c§R6f>—•ÿ [

```
public class frmbase : MonoBehaviour
{
    protected Transform gb => transform.Find("Root");

    public bool isOpen() => gb.gameObject.activeSelf;

    public void show()
    {
        gb.gameObject.SetActive(true);
        if (Application.isPlaying) OnShow();
    }

    public virtual void hide()
    {
        OnHide();
        gb.gameObject.SetActive(false);
    }

    protected virtual void OnShow() { }
    protected virtual void OnHide() { }
}
```

4.2 zà, •béuL—b (frm chapter.cs)

4.2 zà, •béuL—b (frm chapter.cs) •ÇN<NöšqR`R7e°y gamebegin ™—k!R •jy onLevelChange

```
public class frm_chapter : frmbase
{
    public RectTransform chaptercontent;
    public Button btn;
    public Button btnsetup;
    public TextMeshProUGUI staminaText, levelname;

    private void Awake()
    {
        Main.RegistEvent("gamebegin", (x) => {
            brushChapterContent(); show();
            UpdateStaminaDisplay(); return 1;
        });
        Main.RegistEvent("onpowerChange", (x) => {
            UpdateStaminaDisplay(); return null;
        });
        Main.RegistEvent("level_next", (x) => {
            PlayerData.gd.levelid = datamgr.Instance
                .GetLevel(PlayerData.gd.levelid).NextLevel;
            brushChapterContent(); show(); return 1;
        });
        Main.RegistEvent("onChapterChange", (x) => {
            brushChapterContent(); show(); return 1;
        });
        Main.RegistEvent("level_back", (x) => {
            show(); return null;
        });

        btn.onClick.AddListener(() => {
            if (!isTurning())
            {
                int ret = (int)Main.DispEvent(
                    "level_play", PlayerData.gd.levelid);
                if (ret == 1) hide();
            }
        });
        btnsetup.onClick.AddListener(() =>
            Main.DispEvent("show_setup", null));
    }

    private bool isTurning()
    {
        for (int i = 0; i < chaptercontent.childCount; i++)
            if (chaptercontent.GetChild(i)
                .GetComponent<card>().isTurning) return true;
        return false;
    }

    private void brushChapterContent()
    {
        var ch = datamgr.Instance.GetChapter(PlayerData.gd.currChapter);
        var pic = Resources.Load(ch.ChapterFigure) as Texture;
        levelname.text = $"Level {PlayerData.gd.levelid % 10000}";

        // n -deçSarg
        for (int i = chaptercontent.childCount - 1; i >= 0; i--)
            Destroy(chaptercontent.GetChild(i).gameObject);

        Vector2 sz = chaptercontent.rect.size;
        float cw = sz.x / ch.ChapterFigureX;
        float ch2 = sz.y / ch.ChapterFigureY;

        for (int j = 0; j < ch.ChapterFigureY; j++)
        {
            for (int i = 0; i < ch.ChapterFigureX; i++)
            {
                int idx = j * ch.ChapterFigureX + i;
```


4.3.8b. ulv-prb (from game.cs)

```

public class frm_game : frmbase
{
    public picmgr mgr;
    public TextMeshProUGUI level;
    public Button next, back, setup;

    private void Awake()
    {
        Main.RegistEvent("level_play", (x) =>
        {
            if (!PlayerData.gd.hasEnoughpower(10))
            {
                Main.DispEvent("event_msg", "powerN •³");
                return 0;
            }
            PlayerData.gd.m^€ power(10);
            next.gameObject.SetActive(false);

            var leeevel = datamgr.Instance.GetLevel((int)x);
            level.text = $"Level {leevel.Id}";

            if (leevel.DifficultyTier == 2)
            {
                var df = gb.Find("diff");
                df.gameObject.SetActive(true);
                DOVirtual.Float(0, 1, .5f, v =>
                {
                    df.GetComponent<CanvasGroup>().alpha = v
                }.OnComplete(() => DOVirtual.DelayedCall(1, () =>
                {
                    DOVirtual.Float(1, 0, .5f, v =>
                    {
                        df.GetComponent<CanvasGroup>().alpha = v
                    }.OnComplete(() => df.gameObject.SetActive(false)))));
            }
            show();
            StartCoroutine(load(leevel));
            return 1;
        });

        Main.RegistEvent("level_next", (x) => { hide(); return null; });
        next.onClick.AddListener(() => { Main.SendEvent("level_next"); hide(); });
        back.onClick.AddListener(() => { Main.SendEvent("level_back"); hide(); });
        setup.onClick.AddListener(() => Main.DispEvent("show_setup"));
    }

    IEnumerator load(cfg.DrLevel leeevel)
    {
        yield return StartCoroutine(mgr.LoadLevel(leevel));
    }
}

```

4.4k¾•nũŁ•b(ŕm setup.cs)•Npÿ mK<Ōu(• ÿ RŸ€y0

```
public class frm_setup : frmbase
{
    public Button btnInitialize, btnClose, btn_buytili;

    private void Awake()
    {
        btnInitialize.onClick.AddListener(OnInitializeClick);
        btnClose.onClick.AddListener(OnCloseClick);
        btn_buytili.onClick.AddListener(OnBuyTiliClick);
        Main.RegistEvent("show_setup", (x) => { show(); return 1; });
    }

    private void OnBuyTiliClick() => PlayerData.gd.power += 100;
    private void OnCloseClick() => hide();

    private void OnInitializeClick()
    {
        string path = Application.persistentDataPath + "/playerData.json";
        if (File.Exists(path)) File.Delete(path);
        FindObjectOfType<PlayerData>()?.SendMessage("loadData");
        Main.DispEvent("onLevelChange");
        Application.Quit();
    }
}
```

5.epcn'M•n (DrChapter)

DrChapter [šNlZà,,epcnÿ S bì~„%ÈVprG0 R Rr^LR ep0 QsSa ID R ^h0 ••Ç Luban]âQwNÎ Excel [üQún: JSON0

```
[Serializable]
public sealed partial class DrChapter : Luban.BeanBase
{
    public readonly int Id;
    public readonly string ChapterTitle;
    public readonly string ChapterFigure;    // ~„%ÈVp•Dn•T
    public readonly int ChapterFigureX;     // j*T QsSaep
    public readonly int ChapterFigureY;     // ~µT QsSaep
    public readonly int NextChapter;        // N N zàID
    public readonly List<int> LevelId;       // QsSaIDR ^h

    public DrChapter(JSONNode _buf)
    {
        Id = _buf["id"];
        ChapterTitle = _buf["chapter_title"];
        ChapterFigure = _buf["chapter_figure"];
        ChapterFigureX = _buf["chapter_figure_x"];
        ChapterFigureY = _buf["chapter_figure_y"];
        NextChapter = _buf["next_chapter"];
        LevelId = new List<int>();
        foreach (JSONNode e in _buf["level_id"].Children)
            LevelId.Add(e);
    }
}
```

5.2 QsSa'M•n (DrLevel)

DrLevel [šNIQsSaÂepÿ VprG•Dn•0 R Rr^LR 0 -¾^!{I~š0 SŸOMOŸuYep0 füTÈ~••Ÿy»f Vð0

```
[Serializable]
public sealed partial class DrLevel : Luban.BeanBase
{
    public readonly int Id;                // QsSaID
    public readonly int NextLevel;         // N N QsID
    public readonly string LevelFigure;    // VprG•Dn•T
    public readonly int LevelFigureX;     // R ep
    public readonly int LevelFigureY;     // ^Lep
    public readonly int DifficultyTier;    // -¾^! 1=fn• 2=Vð-¾
    public readonly int OutOfPlaceNumber; // g Y SŸOMOŸuY
    public readonly List<int> MDistanceRange; // füTÈ~••Ÿy»f Vð

    public DrLevel(JSONNode _buf)
    {
        Id = _buf["id"];
        NextLevel = _buf["next_level"];
        LevelFigure = _buf["level_figure"];
        LevelFigureX = _buf["level_figure_x"];
        LevelFigureY = _buf["level_figure_y"];
        DifficultyTier = _buf["difficulty_tier"];
        OutOfPlaceNumber = _buf["out_of_place_number"];
        MDistanceRange = new List<int>();
        foreach (JSONNode e in _buf["m_distance_range"].Children)
            MDistanceRange.Add(e);
    }
}
```

5.3 Tables 'M•n-ÆT

Tables IG€Zb@g 'M•n^hÿ ••# JSON R •}TÆ•è^h_u(%œãg•0

```
public partial class Tables
{
    public TbChapter TbChapter { get; }
```


•“Qúÿ [ù^"xŽrG•¹hFf>—•r¶` SÊ•4Vpfôe°0

{—IÖ[ùR ^hQ...kĭN*xŽrGÿ

1. %ãg•T yđf·SÖ•;•WPh (x, y)

2. f·SÖ_SRM[ž–E anchoredPosition

3. W(•“QeR ^hN-gåb~VÛN*e¹T •»\Eÿ WúNŽOM•nÿ – P< 1.5pxÿ

4. hÀgåb~R0v„•»\ET yđf/T&S9'M~„g

5. kcxn•Pc¥ÿ T yđS9'Mÿ !' —•...ĭ•¹ÿ T&R f>y:•¹

6. %Öf>—•ÿ vø•»NûN •¹— %•f>y:R %Öf>y:

7. %Ö•4Vpÿ h9cnVÛN*e¹T kcxn`R cb%Ö/•¹/Sí•l%Ö•4Vp

7.1 pncnc NES

[XhcNå JSON h<_ [XP``NŽ Application.persistentDataPath/playerData.json0 S T+ÿ _SRMQsSa ID0 _SRMzà,, ID0 OSR

7.2 OSR>|û~ß

OSR>N –P 100ÿ kİQsm^€ 100 kİ 15 R "Ÿ•êR``bY 1 p¹0 y»~¿g •ô}/yĩ`bY 0 R)u( LitJson ^•R S [XP``ÿ ••ÇN<Nö onpower

8.1 8.1 Xf

CPU: ARM64 / x86_64 1.5GHz+
Q...[X: 2GB+
[XP": 100MB
\\O^U: 1280×720+

8.2 •oNö%•IB

iOS 13.0+ / Android 8.0+
Unity 2022.3 LTS
.NET Standard 2.1
O••V: DOTween, LitJson, Luban